

Memristor-Based Analog Computation and Neural Network Classification with a Dot Product Engine

Miao Hu, Catherine E. Graves, Can Li, Yunning Li, Ning Ge, Eric Montgomery, Noraica Davila, Hao Jiang, R. Stanley Williams, J. Joshua Yang,* Qiangfei Xia,* and John Paul Strachan*

Using memristor crossbar arrays to accelerate computations is a promising approach to efficiently implement algorithms in deep neural networks. Early demonstrations, however, are limited to simulations or small-scale problems primarily due to materials and device challenges that limit the size of the memristor crossbar arrays that can be reliably programmed to stable and analog values, which is the focus of the current work. High-precision analog tuning and control of memristor cells across a 128×64 array is demonstrated, and the resulting vector matrix multiplication (VMM) computing precision is evaluated. Single-layer neural network inference is performed in these arrays, and the performance compared to a digital approach is assessed. Memristor computing system used here reaches a VMM accuracy equivalent of 6 bits, and an 89.9% recognition accuracy is achieved for the 10k MNIST handwritten digit test set. Forecasts show that with integrated (on chip) and scaled memristors, a computational efficiency greater than 100 trillion operations per second per Watt is possible.

The potential to “compute by physics” in resistive networks through Ohm’s law and Kirchhoff’s current law has been recognized since the early 1950s, with networks designed for wide-ranging applications such as solving partial differential equations,^[1] image filtering,^[2] motion computing,^[3] and neural network algorithms.^[4] This approach was easily overshadowed by the rapid development of complementary metal-oxide semiconductor

(CMOS) transistor-based digital computation. However, general-purpose digital systems for computing are now hitting a wall in energy efficiency with the approaching end of Moore’s law and the growing challenge of the von Neumann communication bottleneck as dataset sizes have exploded. There is also increasing demand for both higher performance in specific applications, such as machine learning and artificial neural networks, and for drastically lower energy consumption in paradigms such as Internet of Things and computing at the Edge.^[5] To accelerate neural network computations, a number of special-purpose and highly efficient architectures have been proposed and developed, both in the digital domain^[6–9] and with mixed analog implementations utilizing emerging nonvolatile technology such as memristors.^[10–12]

The latter are of particular interest in the broader field of neuromorphic computing^[13–15] since memristors offer a highly scalable, high speed, and low power realization of functionality found within biological nervous systems.^[16] This represents a development of memristors for applications beyond high-performance memory^[17] to computational acceleration.

A core computational operation of interest with memristors is vector matrix multiplication (VMM), which can be naturally implemented in dense crossbar geometries in a single-analog computational step utilizing Ohm’s law and Kirchhoff’s current law for summation. This dense memristor crossbar approach for VMM is one example of the “computing by physics” paradigm,^[18] among other recently proposed and developed analog systems.^[19–24] Accelerating VMM computations at lower power through memristor crossbars has the potential to impact important, matrix-heavy applications in speech recognition, image and video classification, neuromorphic engineering, and signal processing. Even more impact comes from utilizing the nonvolatility of memristors to significantly reduce data fetching and communication costs^[11] which fundamentally limits nearly all system performance and is the well-known von Neumann bottleneck.^[25] This bottleneck is particularly pronounced in the training and inference in deep neural networks where the fetching and updating of tens of millions of synaptic weights leads to weeks of computation time and many kilowatts of power consumption limiting the future ability to scale such

Prof. M. Hu, Dr. C. E. Graves, E. Montgomery, Dr. N. Davila,
Dr. R. S. Williams, Dr. J. P. Strachan
Hewlett Packard Labs
Hewlett Packard Enterprise
Palo Alto, CA 94304, USA
E-mail: john-paul.strachan@hpe.com

C. Li, Y. Li, H. Jiang, Prof. J. J. Yang, Prof. Q. Xia
Department of Electrical and Computer Engineering
University of Massachusetts
Amherst, MA 01003, USA
E-mail: jjyang@umass.edu; qxia@umass.edu

Dr. N. Ge
HP Labs
HP Inc.
Palo Alto, CA 94304, USA

 The ORCID identification number(s) for the author(s) of this article can be found under <https://doi.org/10.1002/adma.201705914>.

DOI: 10.1002/adma.201705914

networks to sizes found in human brains ($>10^{15}$ synapses). Crossbars built out of memristors offer a scalable nanotechnology^[26] with broad conductance tunability,^[27] and various bioneurological qualities such as stochasticity and transient plasticity.^[28,29] In order to be implemented practically, demonstrations of sufficient memristor crossbar yield, repeatability, and controllability are required for these targeted applications.

Many recent works have explored using memristor crossbar arrays for computation, particularly in neural network applications.^[30–43] These works considered various memristor technologies, but predominantly chalcogenide phase change memories (PCM) and transition metal-oxide memristors. However, the majority of these works relied heavily on simulations to forecast performance and accuracy for computations within crossbars, without demonstrating actual computations within memristor crossbars. In many cases, the coupled nonlinearities, sneak currents, and other circuit issues were ignored, which are critical to consider when assessing the promise of memristors for computation. The few previous experimental works in full memristor arrays have been limited to small sizes (<1024 memristors), binary device states, or exhibited limitations on reconfigurability. Large neural networks (165 000 synapses) were demonstrated with PCM arrays,^[42] but this work was limited by a sequential interface and could not carry out VMM computations in a single step with access to all word-lines and bit-lines simultaneously, as would be required for an actual computational accelerator. Face recognition with a 128×8 1T1R array was demonstrated with online learning,^[41] and sparse encoding was demonstrated within small memristor arrays.^[40] However, neither work demonstrated or reported computations within the array, nor crucially how the VMM computational accuracy relates to the final performance of the application. The current lack of understanding for how materials, device, and circuit issues relate to a memristor array's computing capability is a major obstacle for designers attempting to simulate and develop complex architectures utilizing memristor crossbar arrays for real computational applications.

In this work, we present a platform for reprogrammable analog computations with integrated transistor-memristor arrays that we call the Dot Product Engine (DPE). Our platform enables exploration of dense VMM computations with arbitrary target matrices converted directly from digital software algorithms. Further, our material stack design for the 1T1M memristor cells enables the precise analog tuning of the memristor conductance over a wide range to implement these arbitrary target matrices faithfully. Our approach differs from online trained implementations^[34,41] in which the derived matrix values are trained specifically for that crossbar array's circuit properties. Instead, our approach enables general acceleration of any matrix operations, taking digital input vectors and matrices, converting into the analog domain for low power, high speed computation, and then providing digital outputs. We demonstrate VMM within memristor arrays up to 128×64 in size at 10 MHz, yielding over 16 000 multiplications and additions in a single clock step, and leading to a forecasted performance of about 115 trillion operations per second per Watt. The resulting VMM outputs have 6 bits of precision, and we demonstrate multiple reprogramming iterations of our memristor arrays for different applications. Finally, we implement a

single-layer neural network to classify the MNIST database of handwritten characters, showing 89.9% recognition accuracy.

The DPE system was designed to perform precise programming of individual memristor cells, reprogramming of the memristor conductance matrix, and, critically, VMM computation in a single step within the memristor crossbar. The VMM computation is performed by applying an input vector of voltages to the rows as brief simultaneously pulses (Ohm's law) and the resulting summed currents (Kirchhoff's law) are collected along the columns (**Figure 1a**). These currents are converted to a voltage (via a transimpedance amplifier, or TIA) and measured after a short delay from the input voltage pulses. The memristor array is programmed so that the individual conductances of the memristor cells comprise the desired computational kernel, and as memristors are nonvolatile, the memristor array maintains the programmed computational kernel. As many applications such as neural network inference do not require frequent reprogramming, the memristor crossbars can be programmed once and then ignored. However, a new application can easily be implemented by simply reprogramming the memristor conductance matrix values and supplying the appropriate inputs to the rows, as we will demonstrate below.

Within the memristor array, we utilize one transistor-one memristor (1T1M) cells in which the transistor limits and controls current during programming to facilitate high accuracy programming but is left fully open in computation mode. The CMOS-memristor integration is carried out in a foundry-compatible back-end-of-the-line (BEOL) process (**Figure 1b**). A transition metal-oxide memristor layer is deposited and patterned atop CMOS access transistors and wiring with 2.0 μm technology. While such 1T1M integration can increase the area compared to purely passive crossbar arrays,^[34] even 1T1M-based architectures can reduce silicon area compared to purely digital approaches.^[11] Passive or 1S1R arrays typically utilized to provide a degree of isolation during programming through device nonlinearity are not suitable for many VMM applications, as the nonlinearity directly invalidates the utilization of Ohm's law and reduces the programming accuracy of analog levels, substantially compromising VMM computation accuracy. Our memristor device stack consists of Ta/HfO₂/Pd (cross-section shown in **Figure 1b**) and has been developed to provide multi-level conductances implemented by continuous tuning of the chemical composition of a Ta-rich conduction channel within the switching matrix.^[44,45] To achieve constant resistance under different voltages (i.e., a linear IV relationship), we selected a resistance range of 1.1–10.0 k Ω (or conductance of 100–900 μS) for accurate analog VMM operation.^[46] With other memristor material systems, we previously demonstrated experimentally that 64 reprogrammable conductance levels (6 bits) can be achieved in individual 1T1M cells.^[27] Here, we extend that work from individual cells to full arrays constituting thousands of cells by programming a 128×64 array to display the Hewlett Packard Enterprise logo and mapping the greyscale logo image to ≈ 180 distinct memristor conductance levels. All programming and computational signals are generated from peripheral printed circuit boards (PCBs) connected to the memristor arrays through probe-card connections (**Figure S1**, Supporting Information), and the design allows for an expandable set of row and column boards that can simultaneously drive (or read)

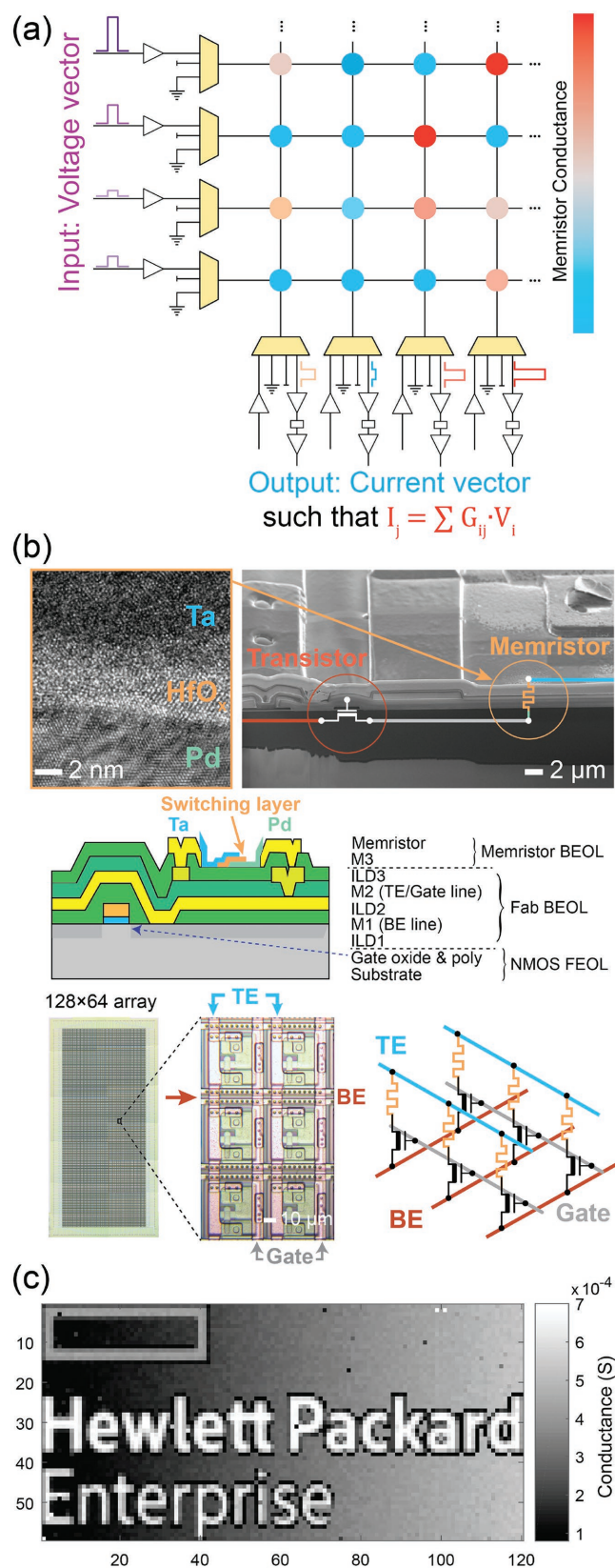


Figure 1. DPE VMM scheme and processing integration of memristor device layers. a) Schematic of DPE circuit operations in a memristor matrix. Desired matrix values written by tuning each vertex's memristor

up to 128 row, 64 column, and 64 selector (transistor gate) lines. The system is designed for fast parallel analog voltage generation and current sensing, with each row independently configurable (Figure 1a) for either voltage driving, floating, or ground, while the columns have the additional capability of current sensing in computation mode. The voltage driving circuit is specifically designed to provide adjustable voltage range (−10 to +10 V) and short, but variable, pulse widths (≈100 ns). The capability of the DPE system to drive and read from multiple columns and rows simultaneously distinguishes the present system from any traditional memory application and enables the present exploration of analog computing applications. This capability also allows for the implementation of array-level conductance programming in large arrays. Table S1 (Supporting Information) gives performance specifications for the entire hardware control system.

Utilizing the DPE platform, we are able to individually access and precisely tune memristor cell conductances (Figure 2a) in an open-loop mode with single pulses (less precise and accurate) or with feedback (more precise and accurate).^[27] The programming algorithm (Figure 2b) is applied across every cell in the entire array with high efficiency. Memristor cells were fabricated with chemically inert bottom row electrodes and reactive top column electrodes, so a positive voltage from row to column induces a RESET operation on the memristor cell (Figure 1b), while a positive voltage from column to row drives a SET operation. A selector line corresponding to the gates of the transistors in a selected column is biased at different voltages to enable different current compliances for the access transistors. All other unselected rows and columns remain floating during individual programming. In Figure 2a, the typical memristor switching curves are obtained by applying a sequence of rising voltage amplitude pulses, with lower voltage (0.2 V) read pulses applied in between to determine the resulting cell conductance (read gate voltage is fixed to 5 V, putting transistors in the linear region). By varying the gate voltage applied during the programming pulses, the maximal current through the device is changed, controlling the final conductance state during SET operations. More details on programming and reading operations are provided in the Supporting Information.

conductance. The desired vector is mapped to applied voltages (purple) along the rows, and the current collected along each column yields the VMM of the memristor conductance matrix G_{ij} and voltage vector V_j . The current from each column is collected by the TIA and converted to a voltage signal, which is held by the sample and hold (S/H) circuit and sensed by an ADC. Each column can be configured to measure current through the TIA, S/H, and ADC path, or apply voltages along the digital-to-analog Converter (DAC) path. b) Memristor device fabrication and processing integration for the 1T1M memristor array. Front-end-of-the-line (FEOL) CMOS transistors and wiring made with 2 μm technology provide the base structure to precisely access individual memristor cells during programming. A foundry-compatible BEOL process integrates the Ta/HfO₂/Pd memristor layer on top of the CMOS. In a 1T1M array, rows share bottom electrode (BE) lines while columns share top electrode (TE) lines and transistor gate lines, with isolation provided by the interlayer dielectric (ILD). c) Demonstration of accurate programming of the 1T1M memristor array with ≈180 conductance levels. The grayscale indicates the memristor cell conductance as given by the scale on right.

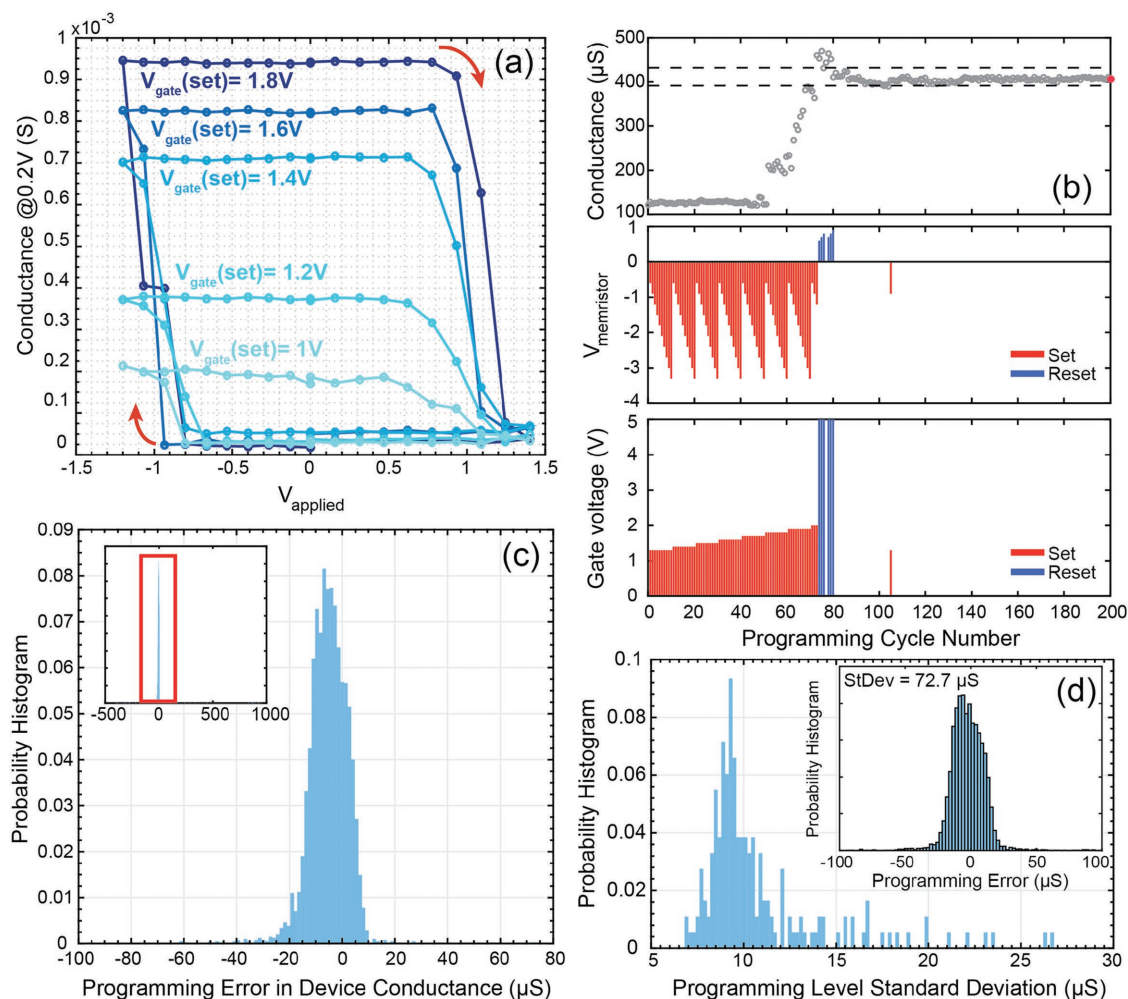


Figure 2. Memristor cell 1T1M properties and tuning capability. a) Memristor switching sweeps with a range of achievable memristor conductance values. Plot shows the conductance (at 0.2 V) following the programming pulse amplitude given by the x axis. By adjusting transistor gate voltage V_{gate} , applied memristor voltage $V_{\text{memristor}}$ and pulse width, the memristor cell can be tuned to different conductance values, shown here for the example of increasing SET gate voltage. b) Example feedback programming algorithm for obtaining a desired memristor conductance (indicated by two black dashed lines in the top panel). For each programming cycle, the algorithm decides whether to apply a SET or RESET operation. The programming algorithm starts by increasing $V_{\text{memristor}}$. If the memristor conductance does not reach the desired target, then V_{gate} is iteratively increased as well. Finally, the pulse width can additionally be increased as necessary. This algorithm will correct for overshooting, or the occasional drift or disturb that can occur while programming other cells in the array. c) Histogram of the programming error (target – final programmed value) for the single-layer neural network weight matrix shown in Figure 3a. d) Histogram of the standard deviation for the set of devices targeted for a particular programming level for the pattern shown in Figure 1c. This shows the spread of values for individual conductance levels in the desired conductance matrix. The inset shows the histogram of programming error (Programmed – Targeted conductance) in μS , similar to that shown in (c). Note that the inset plot is zoomed into the main error peak, while outlier points far off-scale are also present, similar to the inset of (c).

The individual conductance state tuning shown in Figure 2a is systematically applied to all memristor cells sequentially within an array using a feedback algorithm to program the large number of memristors to match the desired conductance matrix G_{target} for a given application. Parallel programming schemes in which a vector of row and column voltages is applied in a single step have been proposed by other groups,^[47] particularly for neural network training, but are not explored here. For a given target conductance and tolerance range, the programming feedback algorithm can vary the applied voltage pulse amplitude and sign, gate voltage, and pulse width (Figure 2b). The top panel of Figure 2b shows the conductance trajectory over 200 programming cycles, the middle panel shows

the changing applied voltage pulse amplitude and sign, and the bottom panel shows the changing gate voltage magnitude. After applying this programming algorithm to every cell in an array, the result is a target pattern^[48] such as those in Figure 1c, Figure 3a, and Figure 4a, demonstrating different applications for images, signal processing, and neural network inference, respectively. There is a trade-off between number of attempted programming cycles and final programming error. In Figure 2, 50 programming cycles were used and the associated errors ($G_{\text{actual}} - G_{\text{target}}$) are plotted as histograms in Figure 2c and the inset of d. As seen, the majority of cells are centered close to zero error, although the inset of (d) yields a standard deviation of 72.7 μS due to outliers beyond the plotted x axis. The

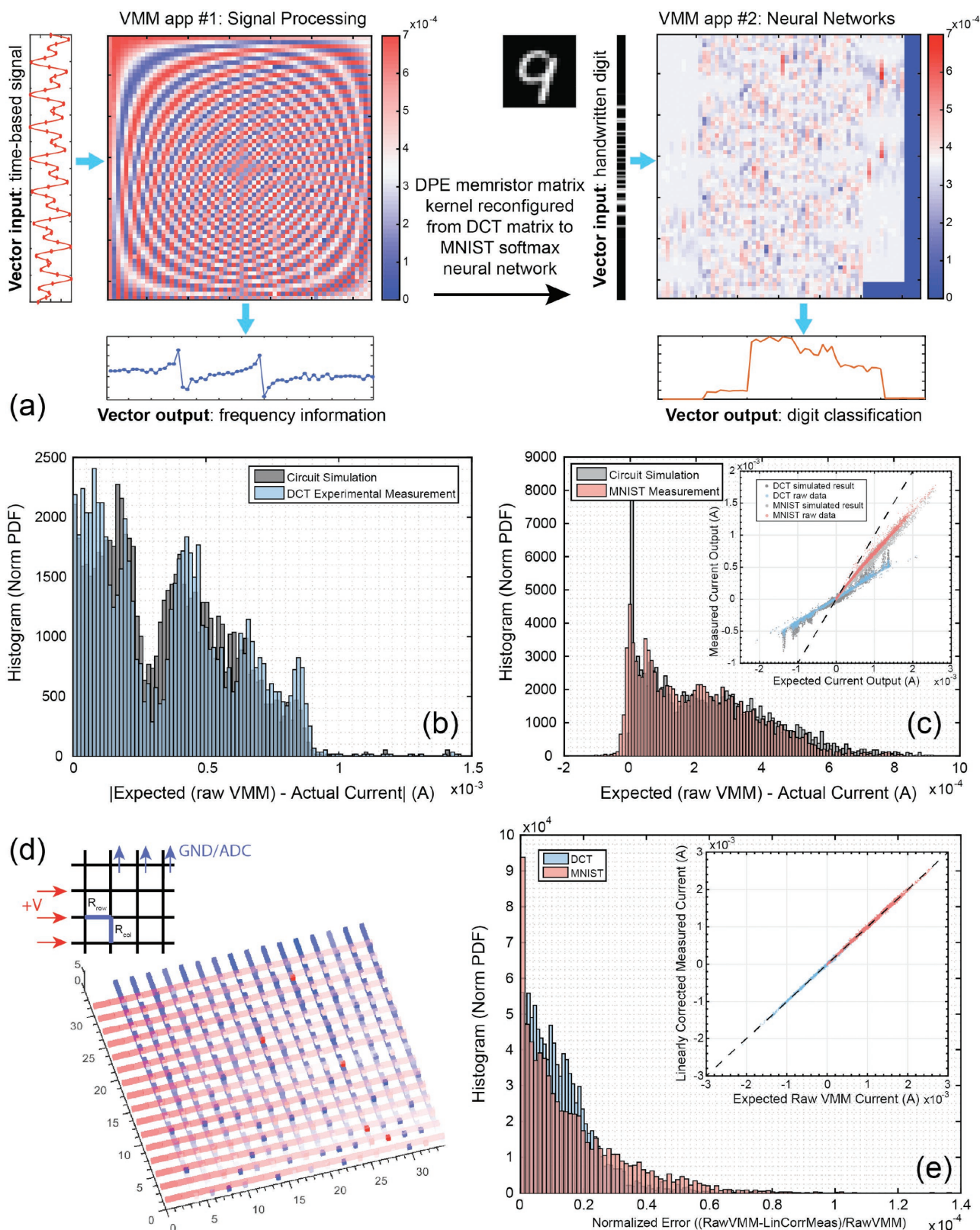


Figure 3. Operation and accuracy of DPE VMM operations for different use cases. a) Two VMM applications programmed and implemented on the same DPE array. First, on the left, a signal processing application was tested using the discrete-cosine transform (DCT) which converts a time-based signal into its frequency components. The same 1T1M memristor array was then reprogrammed for the second application on the right. This second matrix implements a neural network application using a single-layer softmax neural network for recognition of handwritten digits. The DPE VMM output

distribution is not Gaussian, and some cells can remain stuck at either high or low conductances (identified as defects) while others follow a log-normal tail.^[49] However, the spread of individual conductance levels is fairly reasonable, as shown in the main panel of Figure 2d. Here, the histogram of standard deviations is plotted for the different targeted programming levels in the pattern shown in Figure 1c. This effectively gives the spread in conductance for individual target conductance levels in the desired conductance matrix and shows that most levels have a spread $<10 \mu\text{S}$. More details on the array-level feedback tuning algorithms are found in the Supporting Information.

A critical parameter for analog computing is the equivalent digital precision for the computation results. We have quantified these results in our DPE system for different use cases in Figure 3. VMM operations were performed for both a signal processing application of the discrete cosine transform (DCT) as well as neural network inference for the MNIST database (Figure 3a). The DCT application converts a time-based signal into its frequency components and demonstrates full bipolar inputs and matrix values, using two cells per DCT matrix value in order to represent signed numbers.^[46] The neural network application uses only positive inputs and matrix values, implementing a softmax layer for recognition of handwritten digits. Figure 3b shows a histogram of the measured VMM errors compared to ideal expected results. A circuit simulation was also performed (in gray) that takes into account wire resistances and sneak path currents, showing a fairly good match to the experimental results and hence an understanding for the error source which is primarily circuit parasitics. These can be corrected with a linear scaling factor for each column (see below). Figure 3c shows the VMM errors for the neural network application along with the circuit simulations, again showing a close match.

To understand the circuit simulations and the key sources of VMM errors, a visualization for the analog signal deterioration due to parasitics is shown in Figure 3d for a 16×16 array. The color of the middle pillar at each vertex indicates the conductance of the device, and five “stuck on” defects (red dots) are shown. This array is operated by applying voltages on all rows from the left and grounding all columns on top as in the experimental setup. Thus, the signal degrades from left to right (red to white color) and from top to bottom.

To improve the accuracy of our analog VMM computations, each column output is linearly calibrated to account for the above circuit parasitics. The calibration parameters are determined by first running one hundred test input patterns and finding the linear scaling that best matches the expected results. This is a one-time linear tuning of the TIA

circuit in each column, without any performance overhead. When necessary, an additional calibration process can also include compensation to reduce the impact of very high or low conductance defects, as discussed in Note S4 in the Supporting Information. A simple linear scaling reduces errors significantly, as shown in Figure 3e. Here, it is shown that the final error for the DPE system is typically well below $50 \mu\text{S}$ and corresponds to ≈ 6 bits of VMM output accuracy. This is expected to improve further with state-of-the-art fabrication that has higher device yield (fewer defects), lower wire resistances, and by operating memristors at lower conductance states.

Here, we experimentally demonstrate a single-layer neural network inference application with the DPE for handwriting recognition. Neural networks are a key application of interest for acceleration,^[6,11,20,34,38] and this application is characterized by frequent reuse of matrix convolution kernels (and thus an advantage of using memristors by reducing data-fetching), tolerance for defects, and reduced precision requirements.^[50–52] We program a software-trained single-layer network in a 96×40 portion of a 128×64 memristor crossbar for handwritten digit recognition on the full 10 000 MNIST dataset.^[53] To our knowledge, this is the largest demonstration utilizing memristor crossbars in hardware to date. Implementing this single-layer neural network in the DPE platform requires reshaping and partitioning of the neural network weight matrix and input images. A software single-layer network for MNIST classifies 784 pixel inputs (28×28) into 10 possible classes (0 to 9), yielding a network size of 7840 weights. To fit this network into a smaller array capable of supporting 4096 elements, we resize the MNIST images to 19×20 and retrain the weight matrix in software. Ideally (without reshaping), this implementation would use a 380×10 array, but we reshape this into an equivalent 96×40 array. The overall recognition accuracy of the neural network in software did not degrade due to the image resizing, remaining at 92.4%. To perform the digit classification using the DPE, input images are processed by unwrapping the image to a single 1×380 vector (Figure 4a), and then partitioning this into four sets of 96 values (four zeroes are added to the end of the input vector to get the 384 numbers). This yields four sets of pixel values that are converted to voltage signals and applied to the rows of the memristor array, which is programmed to the trained weight matrix (right side of Figure 4a). Only ten output columns are needed for each of the four sets, performing the partial synaptic weight matrix multiplication. A full-image classification is concluded after four such computations, resulting in four 1×10 vectors which are summed, and the resulting digit with the maximal value yields the predicted classification of the

yields the digit classification. b) Histogram of experimentally measured DPE VMM error from the raw mathematical VMM for the DCT application, and comparison results from a circuit simulation (in gray) that takes into account wire resistances and sneak path current. As shown, the circuit simulation reproduces the experimental results quite well, indicating the main source of error of the DPE VMM to a raw mathematical VMM is simply due to circuit parasitics which is easily corrected with a linear scaling factor per column output. c) Same as in (b) but for the neural network application. Inset plots experimentally measured VMM data and circuit simulations versus raw mathematical VMM for both applications. d) Visualized circuit simulation showing circuit nonidealities in a 16×16 array with 5 “stuck on” defects as red vertices. This array is operated by applying voltages on all rows from the left and grounding all columns on top. Therefore, the input signal along the rows degrades left to right (red to white color), and on the columns, you can see the grounding effects also degrading. The color of the middle pillar at each vertex indicates device conductance. e) Histogram of VMM error following implementation of linear scaling factor for each column for DCT and MNIST applications. Inset shows excellent agreement of raw VMM and experimental VMM following a simple linear column scaling.

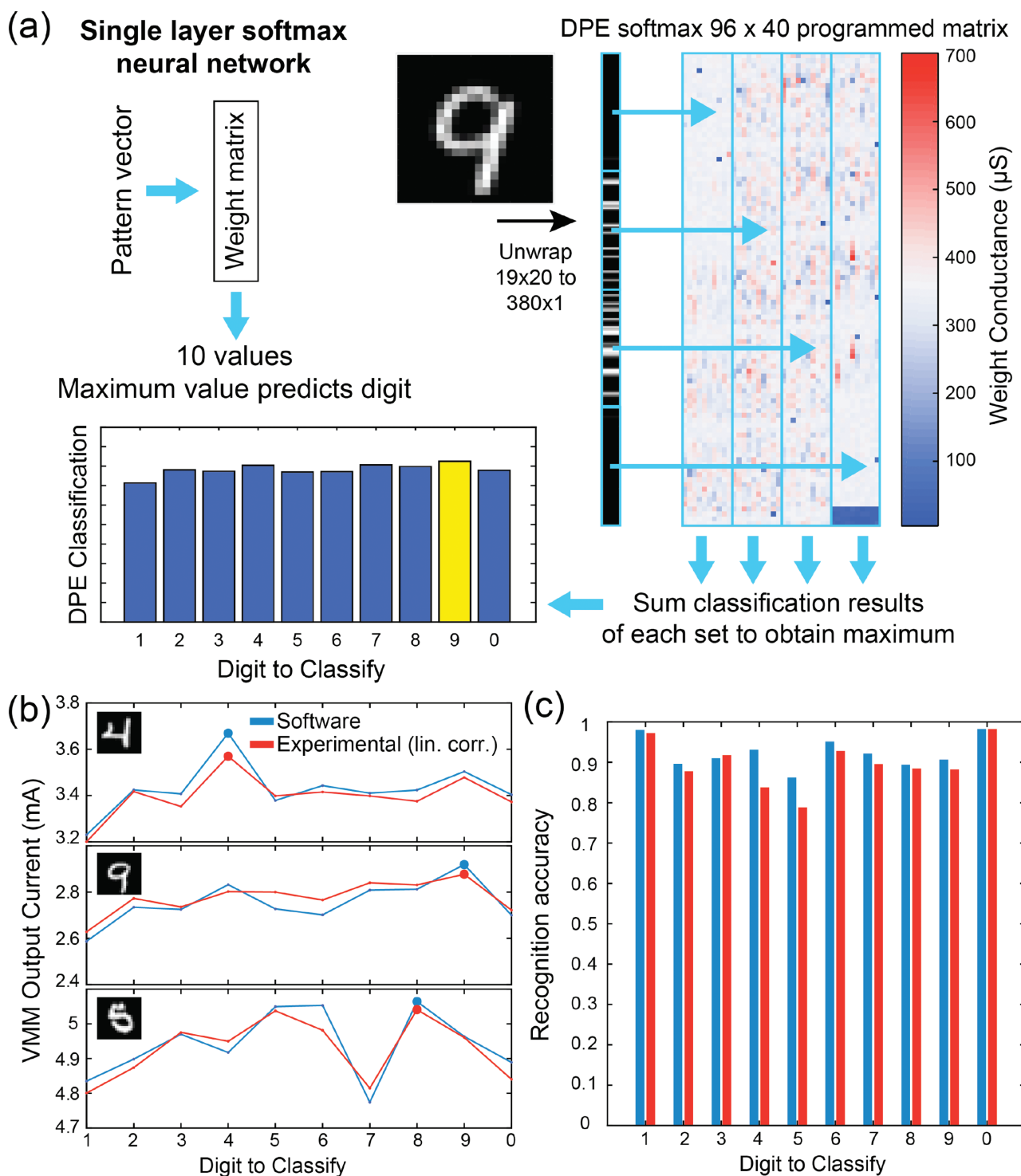


Figure 4. Experimental demonstration of a single-layer neural network for MNIST handwritten digit classification. a) Illustration of the computing procedure. A single-layer softmax neural network is trained offline, converted to target conductance values, and programmed into a 96×40 1T1M crossbar array. For a given input image, the 19×20 pixel image is unwrapped to a 380 input vector, converted to voltages, and applied to the memristor matrix in four sets. The summed result of the four outputs yields the DPE classification. This is illustrated here for the digit “9.” b) Some example classification results of the softmax single-layer neural network on the inset digits “4,” “9,” and “5” with ideal software results (blue) compared to the experimental results (red). c) Total recognition accuracy for each digit for 10 000 images from the MNIST database. Results are shown for the single-layer trained software result compared to the experimental system (same color legend as in (b)). The overall recognition accuracy is 89.9%.

input image. The weight matrix was linearly mapped to the conductance range of the memristor cells from 100 to 700 μS . The experimental output can show nonideal values caused by a combination of small cell-level programming errors as well as stuck ON defects, as can be seen in the output current per column for a few example input patterns (digits 4,9,5) in the 10k MNIST set (Figure 4b). These defects are expected to be significantly reduced in state-of-the-art fabrication facilities.

Our experimental demonstration of inference on the 10k test patterns from MNIST shows the promise of our hardware acceleration. The full recognition accuracy for all 10k testing patterns as a function of the input digit shows that our hardware implementation can even outperform the software-trained network (for digits 3,0, see Figure 4c). With linear correction, the accuracy of the hardware DPE neural network implementation reaches 89.9%, only a 2.5% reduction compared to the ideal software accuracy. This remaining accuracy loss is due to device programming inaccuracies and cumulative finite wire resistances across the array. It is expected that more faithful array programming, fewer memristor cell defects, and especially the use of multilayer neural network implementations will help close the accuracy gap to software levels (see multilayer simulations^[54]). Additionally, using nonlinearity in the activation function would be expected to increase performance further. Our results directly computed the forward inference using existing trained neural networks, showing that these may be utilized directly without redesigning or retraining for the particular hardware. This is well-matched to the paradigms of IoT and Edge computing, in which the goal is to deploy and use trained networks in mobile applications where low power and low cost (small chip area) are key, but speed and performance cannot be compromised. Additionally, in such applications, a reduced precision can often be tolerated. Consequently, a key figure of merit is the computational power efficiency of the system. The present design, including power consumption in the peripheral circuitry, is estimated to yield 115 TOPS W^{-1} (Tera operations per second per Watt, see Table S2 in the Supporting Information).

In this work, we have experimentally validated the potential for analog computing in nonvolatile memristor crossbar arrays. We demonstrated VMM computations in arrays up to 128×64 , supporting the assumptions in architectures showing significant acceleration of neural network and signal processing computations compared to digital implementations.^[11,54,55] We further showed that multiple, stable conductance levels can be realized in 1T1M cells composed of Ta/HfO₂ memristors with 6 bits of precision. Given the low-precision requirements in many applications,^[50–52] this exceeds the demands considerably. We showed that large memristor crossbars can carry out single-step VMM operations in-memory leading to high-throughput and low-energy consumption. The computing accuracy was shown to be acceptable for machine learning applications including image recognition, and we also demonstrated reprogrammability for multiple applications. Direct implementation of the forward inference of a neural network for MNIST image recognition was shown, yielding 89.9% accuracy for a single layer. Reducing stuck-ON cells and using multilayer neural networks are expected to increase the accuracy to state of the art.^[56]

A key question is how the power efficiency of such an analog DPE system compares to the state of the art, both in performing VMM computations and more broadly in convolutional neural network applications. This requires forecasting the DPE when implemented in an integrated chip and using scaled technology nodes. Although the present system operates at 10 MHz, this is due to the larger parasitics of 2 μm technology and probecard interfaces. By assuming <100 nm technology and integrated control electronics, an operating frequency of 150 MHz is easily achievable, along with analog–digital converters (ADCs) operating at <133 μW ^[57] per column channel. Given the 16 320 multiplications and additions performed in a 128×64 array, this leads to a computational efficiency of 115 TOPS W^{-1} (Tera Operations per Second per Watt). In comparison, digital technology performing the same VMM operations at only 4-bit accuracy using 40 nm CMOS is estimated to operate at 7 TOPS W^{-1} .^[40] In addition, the high power and area cost of ADCs could be alleviated by only applying ADCs to final outputs of multilayer neural networks. For a large family of neural network algorithms, ADCs could also be replaced with simple circuits such as threshold gates, comparators, or amplifiers, even further reducing DPE power and area.

The above shows the power efficiency in replacing digital VMM blocks with analog, memristor-based DPE circuits. An important question is whether such computational efficiency is maintained at the application level, for example in full convolutional neural network (CNN) inference where a streaming input of images is classified. In an earlier architectural study and performance estimation,^[11] we evaluated CNN applications computed by a DPE-based system composed of many 128×128 arrays, each with only 2 bits per cell and a 10 MHz operating frequency. All ADC elements, data routing, and buffering are taken into account, along with the costs in breaking down the larger images across multiple arrays. The results show a 14.8x, 5.5x, and 7.5x improvement in throughput (inferences per second), energy, and computational density (inferences per second per chip area) over the leading Digital ASIC implementation^[6] for the same task. Thus, the present work experimentally validates the assumptions for that architectural study, providing a base-line for future analog computing systems and the potential to accelerate and significantly lower energy consumption for important applications.

Experimental Section

Transistor and Memristor Integration: An array of NMOS transistors were fabricated in a commercial fab with low wire resistance. The transistors had a feature size of 2 μm . The memristor arrays were fabricated in a university clean room aligned to the underlying transistors following an argon plasma to remove protective metal-oxide layers. Photolithography patterning was used, along with thin-film deposition and liftoff. Sputter deposition of 5 nm silver (Ag) and 200 nm palladium (Pd) was used for the metal vias. After lifting-off in warm acetone, the sample was annealed at 300 °C for 30 min in nitrogen with a flow of 20 sccm. A 60 nm Pd layer with a 5 nm tantalum (Ta) adhesive layer was sputtered to serve as the bottom electrode. A 5 nm HfO₂ switching layer was deposited by atomic layer deposition using water and tetrakis(dimethylamido)hafnium as precursors at 250 °C, to ensure high film quality and step coverage. The patterning of the switching layer was done by photolithography and reactive ion etch

using CHF₃/O₂ chemistry. Sputter deposited Ta of 50 nm thickness followed by 10 nm Pd was used in a lift-off process to serve as the top electrode.

Control Electronics and Chip Interface: A dedicated system of PCB was designed and manufactured to accomplish all memristor state programming (reading and writing) as well as computational operations. The novel operating mode for computations involved simultaneous driving of all rows in the array along with simultaneous reading of the currents in all columns. The designed PCB system was able to perform these operations with driving pulses from 164 ns to greater than 4 μ s. Table S1 (Supporting Information) had detailed specifications.

Interfacing of the control PCB system to the memristor arrays was accomplished on a probe station with a specially designed cantilever probecard system for high-speed signal transmission and the flexibility to test and operate new memristor chips without the need for wire-bonding. Images of the setup were available in the Supporting Information. All system configurations and final result readout were performed by a control computer communicating with a microcontroller in the PCB system. The microcontroller supported a small DPE instruction set, receiving configuration instructions to control the column and row boards for different write or read operations at arbitrary positions in the crossbar array. Communication with the computer limited all operation times, and to achieve higher speed and efficiency, instructions for fully array-level read and write operations were also implemented in the firmware. In this way, for example, one array-level read command by the firmware would read all devices in an array (more than 8000 devices) in less than 2 s. A graphical user interface, data visualization scripts, programming scripts, and computing scripts were developed to support all operations described in this work.

Single-Layer Neural Network for MNIST Classification: The single-layer neural network was trained in MATLAB using softmax regression. Softmax is a simple and well-suited model for MNIST pattern recognition since there were only ten possible classifications. 60k images from the MNIST database were used for training, employing gradient descent to find the target synaptic weights related to the cost function defined in softmax regression. When the trained network was used for classification, outputs were exponentiated and normalized to generate a probability distribution. This last step was not implemented in the DPE since the present memristor crossbars would not directly compute the nonlinear function.

Supporting Information

Supporting Information is available from the Wiley Online Library or from the author.

Acknowledgements

The authors acknowledge fruitful discussions with Mark McLean, David Mountain, and Richard Slusher. This research was based upon work supported by the Office of the Director of National Intelligence (ODNI), Intelligence Advanced Research Projects Activity (IARPA), via contract number 2014-14080800008. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the ODNI, IARPA, or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright annotation thereon.

Conflict of Interest

The authors declare no conflict of interest.

Keywords

crossbar arrays, memristor, metal oxide, neuromorphic computing

Received: October 11, 2017
Published online: January 10, 2018

- [1] G. Liebmann, *Br. J. Appl. Phys.* **1950**, 1, 92.
- [2] H. Kobayashi, J. L. White, A. A. Abidi, *IEEE J. Solid-State Circuits* **1991**, 26, 738.
- [3] J. Hutchinson, C. Koch, J. Luo, C. Mead, *Computer* **1988**, 21, 52.
- [4] H. P. Graf, L. D. Jackel, *IEEE Circuits Devices Mag.* **1989**, 5, 44.
- [5] R. S. Williams, *Comput. Sci. Eng.* **2017**, 19, 7.
- [6] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, O. Temam, in *Proc. 47th Annual IEEE/ACM Int. Symp. on Microarchitecture*, IEEE, Piscataway, NJ, USA **2014**, pp. 609–622.
- [7] Y.-H. Chen, J. Emer, V. Sze, in *Proc. 43rd Int. Symp. on Computer Architecture*, (ISCA), IEEE, Piscataway, NJ, USA **2016**, pp. 367–379.
- [8] B. Reagen, P. Whatmough, R. Adolf, S. Rama, H. Lee, S. K. Lee, J. M. Hernández-Lobato, G.-Y. Wei, D. Brooks, in *Proc. 43rd Int. Symp. on Computer Architecture*, (ISCA), IEEE, Piscataway, NJ, USA **2016**, pp. 267–278.
- [9] S. Liu, Z. Du, J. Tao, D. Han, T. Luo, Y. Xie, Y. Chen, T. Chen, in *Proc. 43rd Int. Symp. on Computer Architecture*, (ISCA), IEEE, Piscataway, NJ, USA **2016**, pp. 393–405.
- [10] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, Y. Xie, in *Proc. 43rd Int. Symp. on Computer Architecture*, IEEE Press, Piscataway, NJ, USA **2016**, pp. 27–39.
- [11] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramanian, J. P. Strachan, M. Hu, R. S. Williams, V. Srikumar, in *Proc. 43rd Int. Symp. on Computer Architecture*, (ISCA), IEEE, Piscataway, NJ, USA **2016**, pp. 14–26.
- [12] T. Gokmen, Y. Vlasov, *Front. Neurosci.* **2016**, 10.
- [13] C. Mead, *Proc. IEEE* **1990**, 78, 1629.
- [14] G. Indiveri, B. Linares-Barranco, T. J. Hamilton, A. Van Schaik, R. Etienne-Cummings, T. Delbruck, S.-C. Liu, P. Dudek, P. Häfliger, S. Renaud, J. Schemmel, G. Cauwenberghs, J. Arthur, K. Hynna, F. Folowosele, S. Saighi, T. Serrano-Gotarredona, J. Wijekoon, Y. Wang, K. Boahen *Front. Neurosci.* **2011**, 5, 73.
- [15] T. Hasegawa, K. Terabe, T. Tsuruoka, M. Aono, *Adv. Mater.* **2012**, 24, 252.
- [16] M. P. Sah, H. Kim, L. O. Chua, *IEEE Circuits Syst. Mag.* **2014**, 14, 12.
- [17] J. Y. Seok, S. J. Song, J. H. Yoon, K. J. Yoon, T. H. Park, D. E. Kwon, H. Lim, G. H. Kim, D. S. Jeong, C. S. Hwang, *Adv. Funct. Mater.* **2014**, 24, 5316.
- [18] K. Steinbuch, *Biol. Cybern.* **1961**, 1, 36.
- [19] R. Chawla, A. Bandyopadhyay, V. Srinivasan, P. Hasler, in *Proc. IEEE 2004 Custom Integrated Circuits Conf.*, IEEE, Piscataway, NJ, USA **2004**, pp. 651–654.
- [20] J. Hasler, H. B. Marr, *Front. Neurosci.* **2013**, 7, 118.
- [21] K. K. Likharev, *Sci. Adv. Mater.* **2011**, 3, 322.
- [22] D. B. Strukov, K. K. Likharev, *Nanotechnology* **2005**, 16, 888.
- [23] N. Ge, J. H. Yoon, M. Hu, E. J. Merced-Grafals, N. Davila, J. P. Strachan, Z. Li, H. Holder, Q. Xia, R. S. Williams, X. Zhou, J. J. Yang, *Sci. Rep.* **2017**, 7, 40135.
- [24] S. Kumar, J. P. Strachan, R. S. Williams, *Nature* **2017**, 548, 318.
- [25] J. Backus, *Commun. ACM* **1978**, 21, 613.
- [26] S. Pi, P. Lin, Q. Xia, *J. Vac. Sci. Technol., B: Nanotechnol. Microelectron.: Mater., Process., Meas., Phenom.* **2013**, 31, 06FA02.
- [27] E. J. Merced-Grafals, N. Dávila, N. Ge, R. S. Williams, J. P. Strachan, *Nanotechnology* **2016**, 27, 365202.
- [28] C. Du, W. Ma, T. Chang, P. Sheridan, W. D. Lu, *Adv. Funct. Mater.* **2015**, 25, 4290.

- [29] Z. Wang, S. Joshi, S. E. Savel'ev, H. Jiang, R. Midya, P. Lin, M. Hu, N. Ge, J. P. Strachan, Z. Li, Q. Wu, M. Barnell, G.-L. Li, H. L. Xin, R. S. Williams, Q. Xia, J. J. Yang, *Nat. Mater.* **2017**, *16*, 101.
- [30] M. Suri, O. Bichler, D. Querlioz, O. Cueto, L. Perniola, V. Sousa, D. Vuillaume, C. Gamrat, B. DeSalvo, in *Proc. 2011 IEEE Int. Electron Devices Meeting (IEDM)*, IEEE, Piscatawy, NJ, USA **2011**, p. 4.
- [31] S. Yu, B. Gao, Z. Fang, H. Yu, J. Kang, H.-S. P. Wong, in *Proc. 2012 IEEE Int. Electron Devices Meeting (IEDM)*, IEEE, Piscatawy, NJ, USA **2012**, pp. 10–14.
- [32] G. W. Burr, P. Narayanan, R. M. Shelby, S. Sidler, I. Boybat, C. di Nolfo, Y. Leblebici, in *Proc. 2015 IEEE Int. Electron Devices Meeting (IEDM)*, IEEE, Piscatawy, NJ, USA **2015**, p. 4.
- [33] G. W. Burr, R. M. Shelby, A. Sebastian, S. Kim, S. Kim, S. Sidler, K. Virwani, M. Ishii, P. Narayanan, A. Fumarola, L. L. Sanches, I. Boybat, M. Le Gallo, K. Moon, J. Woo, H. Hwang, Y. Leblebici, *Adv. Phys.: X* **2017**, *2*, 89.
- [34] M. Prezioso, F. Merrikh-Bayat, B. D. Hoskins, G. C. Adam, K. K. Likharev, D. B. Strukov, *Nature* **2015**, *521*, 61.
- [35] P. M. Sheridan, C. Du, W. D. Lu, *IEEE Trans. Neural Netw. Learn. Syst.* **2016**, *27*, 2327.
- [36] S. Ambrogio, S. Balatti, V. Milo, R. Carboni, Z.-Q. Wang, A. Calderoni, N. Ramaswamy, D. Ielmini, *IEEE Trans. Electron Devices* **2016**, *63*, 1508.
- [37] S. Yu, Z. Li, P.-Y. Chen, H. Wu, B. Gao, D. Wang, W. Wu, H. Qian, in *Proc. 2016 IEEE Int. Electron Devices Meeting (IEDM)*, IEEE, Piscatawy, NJ, USA **2016**, pp. 12–16.
- [38] S. Agarwal, S. J. Plimpton, D. R. Hughart, A. H. Hsia, I. Richter, J. A. Cox, C. D. James, M. J. Marinella, in *Proc. 2016 Int. Joint Conf. on Neural Networks, (IJCNN)*, IEEE, Piscatawy, NJ, USA **2016**, pp. 929–938.
- [39] G. Indiveri, E. Linn, S. Ambrogio, *Resistive Switching: From Fundamentals of Nanoionic Redox Processes to Memristive Device Applications*, (Eds: D. Ielmini, R. Waser), Wiley-VCH, Weinheim, Germany **2016**, pp. 715–736.
- [40] P. M. Sheridan, F. Cai, C. Du, W. Ma, Z. Zhang, W. D. Lu, *Nat. Nanotechnol.* **2017**, *12*, 784.
- [41] P. Yao, H. Wu, B. Gao, S. B. Eryilmaz, X. Huang, W. Zhang, Q. Zhang, N. Deng, L. Shi, H.-S. P. Wong, H. Qian, *Nat. Commun.* **2017**, *8*, 15199.
- [42] G. W. Burr, R. M. Shelby, S. Sidler, C. di Nolfo, J. Jang, I. Boybat, R. S. Shenoy, P. Narayanan, K. Virwani, E. U. Giacometti, B. N. Kurdi, H. Hwang, *IEEE Trans. Electron Devices* **2015**, *62*, 3498.
- [43] S. Yu, P. Y. Chen, Y. Cao, L. Xia, Y. Wang, H. Wu, in *Proc. 2015 IEEE Int. Electron Devices Meeting (IEDM)*, IEEE, Piscatawy, NJ, USA **2015**, pp. 17.3.1–17.3.4.
- [44] A. Wedig, M. Luebben, D.-Y. Cho, M. Moors, K. Skaja, V. Rana, T. Hasegawa, K. K. Adeplli, B. Yildiz, R. Waser, I. Valov, *Nat. Nanotechnol.* **2015**, *11*, 67.
- [45] H. Jiang, L. Han, P. Lin, Z. Wang, M. H. Jang, Q. Wu, M. Barnell, J. J. Yang, H. L. Xin, Q. Xia, *Sci. Rep.* **2016**, *6*, 28525.
- [46] C. Li, M. Hu, Y. Li, H. Jiang, N. Ge, E. Montgomery, J. Zhang, W. Song, N. Davila, C. E. Graves, Z. Li, J. P. Strachan, P. Lin, Z. Wang, M. Barnell, Q. Wu, R. S. Williams, J. J. Yang, Q. Xia, *Nat. Electron.* **2017**, <https://doi.org/10.1038/s41928-017-0002-z>.
- [47] S. Agarwal, T.-T. Quach, O. Parekh, A. H. Hsia, E. P. DeBenedictis, C. D. James, M. J. Marinella, J. B. Aimone, *Front. Neurosci.* **2016**, *9*, 484.
- [48] K.-H. Kim, S. Gaba, D. Wheeler, J. M. Cruz-Albrecht, T. Hussain, N. Srinivasa, W. Lu, *Nano Lett.* **2012**, *12*, 389.
- [49] G. Medeiros-Ribeiro, F. Perner, R. Carter, H. Abdalla, M. D. Pickett, R. S. Williams, *Nanotechnology* **2011**, *22*, 95702.
- [50] M. Courbariaux, Y. Bengio, J.-P. David, presented at *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA May **2015**.
- [51] M. Rastegari, V. Ordonez, J. Redmon, A. Farhadi, in *Proc. European Conf. on Computer Vision*, Springer, Cham, Switzerland **2016**, pp. 525–542.
- [52] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, Y. Zou, *arXiv Prepr. arXiv1606.06160* **2016**.
- [53] Y. LeCun, C. Cortes, C. J. C. Burges, *The MNIST Database of Handwritten Digits*, National Institute of Standards and Technology, Gaithersburg, MD, USA **1998**.
- [54] M. Hu, J. P. Strachan, Z. Li, E. M. Grafals, N. Davila, C. Graves, S. Lam, N. Ge, J. J. Yang, R. S. Williams, in *Proc. 2016 53rd ACM/EDAC/IEEE Design Automation Conf., (DAC)*, Association for Computing Machinery, New York **2016**, pp. 1–6.
- [55] M. Hu, J. P. Strachan, in *Proc. IEEE Int. Conf. on Rebooting Computing (ICRC)*, IEEE, Piscatawy, NJ, USA **2016**, pp. 1–5.
- [56] C. Liu, M. Hu, J. P. Strachan, H. Li, in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, Association for Computing Machinery, New York **2017**, pp. 1–6.
- [57] G. Van Der Plas, B. Verbruggen, in *Proc. 2008 IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Pap.*, IEEE, Piscatawy, NJ, USA **2008**, pp. 242–610.